

 Cette page a été traduite à partir de l'anglais par la communauté. [Vous pouvez contribuer en rejoignant la communauté francophone sur MDN Web Docs.](#)

[View in English](#)

Always switch to English

Introduction à Django

[Aperçu : Django Web Framework \(Python\)](#)[Suivant →](#)

Dans ce premier article sur Django, nous allons répondre à la question suivante "Qu'est ce que Django ?", et nous vous montrerons en quoi cette infrastructure d'application (aussi appelée framework) est spéciale. Nous allons ensuite voir les principales fonctionnalités, mais aussi quelques fonctionnalités avancées que nous n'aurons malheureusement pas le temps de voir en détails dans ce module. Nous allons aussi vous montrer comment fonctionne une application Django (bien que nous n'ayons pas d'environnement où le tester) .

Prérequis:	Connaissances basiques de programmation. Une compréhension générale de la programmation coté serveur ainsi qu'une compréhension des interactions client-serveur dans les sites web .
Objectif:	Se familiariser avec Django en comprenant ce que c'est, les fonctionnalités qu'il fournit ainsi que les blocs de construction principaux d'une application Django.

Qu'est ce que Django?

Django est un framework Python de haut niveau, permettant un développement rapide de sites internet, sécurisés, et maintenables. Créé par des développeurs expérimentés, Django prend en charge la plupart des tracas du développement web, vous pouvez donc vous concentrer sur l'écriture de votre application sans avoir besoin de réinventer la roue. Il est gratuit, open source, a une communauté active, une bonne documentation, et plusieurs options pour du support gratuit ou non.

Django vous aide à écrire une application qui est:

Complète

Django suit la philosophie "Piles incluses" et fournit presque tout ce que les développeurs pourraient vouloir faire. Comme tout ce dont vous avez besoin est une partie de ce "produit", tout fonctionne parfaitement ensemble, suivant des principes de conception cohérents, il possède également une [documentation complète](#)  et à jour.

Polyvalent

Django peut être (et a été) utilisé pour créer presque tous les genres de sites — du gestionnaire de données aux wikis, jusqu'aux réseaux sociaux et aux sites d'actualités. Il peut fonctionner avec n'importe quelle infrastructure côté client, et peut renvoyer des données dans quasiment n'importe quel format (notamment HTML, RSS, JSON, XML, etc). Le site sur lequel vous lisez en ce moment est basé sur Django!

Tandis qu'il fournit presque toutes les fonctionnalités dont vous pourriez avoir besoin (comme des bases de données populaires, des moteurs de modélisation, etc.), il peut tout de même être étendu pour utiliser d'autres composants si besoin.

Sécurisé

Django aide les développeurs à éviter les erreurs de sécurité classique en fournissant une infrastructure conçue pour "faire ce qu'il faut" pour protéger les sites internet automatiquement. Par exemple, Django fournit un moyen sécurisé pour gérer les comptes des utilisateurs ainsi que leurs mots de passe, évitant les erreurs classiques comme mettre des informations sur la session dans des cookies, où elles sont vulnérables (à la place les cookies contiennent seulement une clé, et les données sont stockées dans la base de données), ou directement stocker des mots de passe, au lieu de mots de passe hachés.

Un mot de passe haché est une valeur dont la longueur est fixée, créée en envoyant le mot de passe à travers une [fonction de hachage cryptographique](#). Django peut vérifier si un mot de passe entré est correct en l'envoyant dans la fonction de hachage et en comparant le retour avec la valeur stockée dans la base de données. De ce fait, la nature unidirectionnelle de la fonction rend difficile pour un attaquant de retrouver le mot de passe d'origine, même si la valeur hachée est compromise.

Django active par défaut la protection contre beaucoup de vulnérabilités, comme les injections SQL, le cross-site scripting, le cross-site request forgery et le clickjacking (voir [Website security](#) pour plus de détails sur ce genre d'attaques).

Scalable

Django utilise une architecture composite "shared-nothing" (chaque composant de l'architecture est indépendant des autres, et peut ainsi être remplacé ou changé si besoin). En ayant des séparations nettes entre les différentes parties, Django peut se scaler lors d'une hausse de trafic en ajoutant du hardware à tous les niveaux : serveurs cache, serveurs de base de données, serveurs d'application. Certains des sites les plus fréquentés ont réussi à scaler Django pour répondre à leur demande (par exemple, Instagram et Disqus pour ne nommer qu'eux deux).

Maintenable

Les principes de design du code Django encouragent la création d'un code simple à maintenir et réutilisable. Il fait notamment appel à la philosophie du Ne Vous Répétez Pas (DRY pour Don't Repeat Yourself en anglais), afin d'éviter toute duplication superflue, réduisant la taille de votre code. Django promeut aussi le regroupement de fonctionnalités reliées entre elles en "applications" réutilisables et, à un plus bas niveau, regroupe des lignes de code dépendantes entre elles en modules (suivant les lignes du motif d'architecture Modèle-vue-contrôleur (MVC)).

Portable

Django est écrit en Python, qui fonctionne sous diverses plateformes. Cela veut dire que vous ne serez plus contraint par une plateforme en particulier, et vous pourrez faire fonctionner vos applications sous autant de versions de Linux, Windows et Mac OS X que vous le souhaitez. De plus, Django est très bien supporté par plusieurs fournisseurs d'hébergement web, qui offrent souvent des infrastructures et de la documentation spécifiques pour héberger des sites Django.

D'où vient-il ?

À l'origine, Django fut développé entre 2003 et 2005 par une équipe web responsable de la création et de la maintenance de sites journalistiques. Après avoir créé un certain nombre de sites, l'équipe commença à exclure et à réutiliser des codes récurrents et des schémas d'architecture. Ce code récurrent finit par évoluer en un framework générique de développement web, qui fut mis à disposition en open-source sous le projet "Django" en Juillet 2005.

Django a continué à se développer et à s'améliorer, depuis sa première sortie (1.0) en Septembre 2008 jusqu'à la version 2.0 récemment sortie (2017). Chaque sortie a ajouté son lot de nouvelles fonctionnalités et de corrections de bugs, allant du support de nouveaux types de bases de données, de moteurs de templates et de cache, à l'addition de fonctions et de classes de vues 'génériques' (qui réduisent la quantité de code que doivent écrire les développeurs pour tout un tas de tâches de programmation).

 **Note :** Consultez les [notes de publication](#) sur le site web de Django pour voir les changements apportés dans les versions récentes, ainsi que tout le travail accompli pour améliorer Django.

Désormais, Django est un projet open-source collaboratif florissant, avec plusieurs milliers d'utilisateurs et de contributeurs. Bien que plusieurs fonctionnalités reflètent encore ses origines, Django a évolué en un framework versatile capable de développer n'importe quel type de site web.

À quel point Django est-il populaire ?

Il n'y a pas encore de mesure toute prête et définitive de la popularité des frameworks orientés serveur (bien que des sites comme [Hot Frameworks](#) tentent d'estimer cette popularité en utilisant des moyens comme le comptage de projets GitHub et de questions sur StackOverflow pour chaque plateforme). Une meilleure question serait plutôt est-ce que Django est "suffisamment populaire" pour éviter les problèmes des plateformes moins populaires. Va-t-il continuer d'évoluer ? Pourrez-vous obtenir de l'aide si vous en avez besoin ? Aurez-vous des opportunités d'emploi si vous apprenez Django ?

Si l'on se base sur la quantité de sites web reconnus qui utilisent Django, la quantité de personnes contribuant à son code source, et la quantité de personnes fournissant du support libre ou payant, alors oui, Django est un framework populaire !

Parmi les sites web qui utilisent Django, on retrouve : Disqus, Instagram, la Knight Foundation, la MacArthur Foundation, Mozilla, National Geographic, l'Open Knowledge Foundation, Pinterest et Open Stack (source : [Page d'accueil de Django](#)).

Django est-il restrictif ?

Les frameworks web sont souvent qualifiés de "restrictifs" ou de "non-restrictifs".

Les frameworks restrictifs sont ceux vous donnant une "bonne façon" de réaliser une tâche particulière. Ils sous-tendent souvent un développement rapide *dans un domaine particulier* (résoudre des problèmes d'un type particulier), car cette bonne façon de faire est souvent bien comprise et bien documentée. Cependant, ils peuvent être moins flexibles dans leurs capacités à résoudre des problèmes en dehors de leur domaine, et offrent souvent moins de choix sur les composants et approches utilisables.

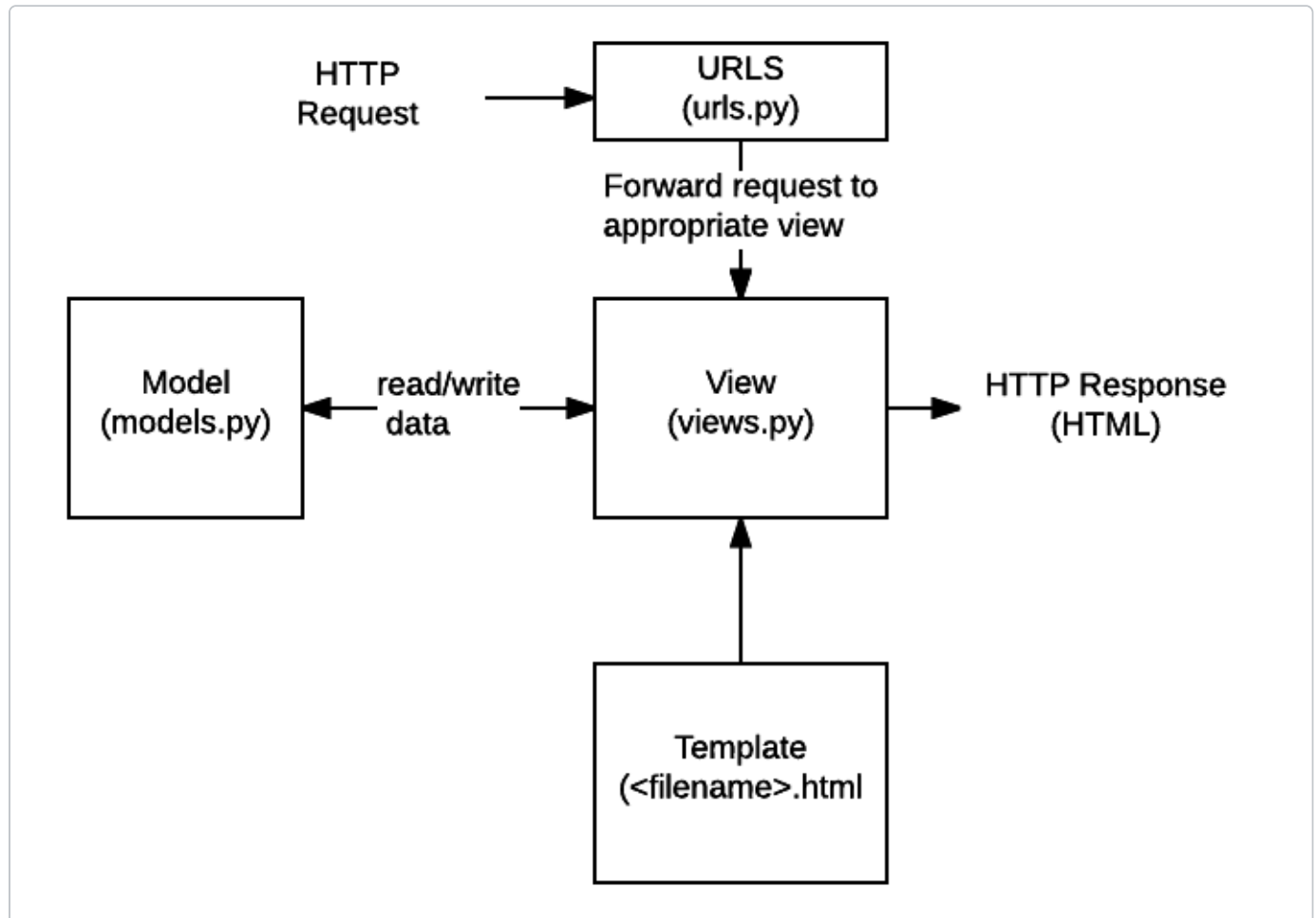
En contraste, les frameworks non-restrictifs ont moins de restrictions sur la meilleure façon d'assembler des composants ensemble pour achever un but, voire même sur quels composants utiliser. Ils simplifient la tâche des développeurs en leur permettant d'utiliser les outils les mieux adaptés à la réalisation d'une tâche particulière, au coût toutefois du besoin du développeur de trouver ces composants.

Django est "plus ou moins restrictif", et offre ainsi le "meilleur de chaque approche". Il fournit un ensemble de composants pour gérer la plupart des tâches de développement web ainsi qu'une (ou deux) approches préférées sur leur utilisation. Toutefois, l'architecture découplée de Django implique que vous pouvez généralement choisir parmi un certain nombre d'options différentes, ou bien fournir un support pour des approches complètement nouvelles si vous le désirez.

À quoi ressemble le code Django ?

Dans un site web traditionnel orienté-données, une application web attend une requête HTTP d'un navigateur web (ou tout autre client). Quand une requête est reçue, l'application en comprend les besoins d'après l'URL et parfois d'après les informations en `POST` data ou `GET` data. En fonction de ce qui est attendu, elle peut ensuite lire ou écrire l'information dans une base de données ou réaliser une autre tâche requise pour satisfaire la requête. L'application renvoie ensuite une réponse au navigateur web, créant souvent en dynamique une page HTML affichée dans le navigateur où les données récupérées sont insérées dans les balises d'un modèle HTML.

Les applications web Django regroupent généralement le code qui gère chacune de ces étapes dans des fichiers séparés :



- **URLs** : Bien qu'il soit possible de traiter les requêtes de chaque URL via une fonction unique, il est bien plus viable d'écrire une fonction de vue isolée qui gèrera chaque ressource. Un mapper URL est utilisé pour rediriger les requêtes HTTP à la vue appropriée d'après l'URL de requête. Le mapper URL peut aussi faire la correspondance entre des patterns de chaînes de caractères ou de nombres qui apparaissent dans une URL et passer ces derniers comme données dans une fonction de vue.
- **Vues** : Une vue est une fonction de gestion des requêtes, qui reçoit des requêtes HTTP et renvoie des réponses HTTP. Les vues accèdent aux données requises pour satisfaire des requêtes via des *modèles*, et délèguent le formatage des réponses aux *templates*.
- **Modèles** : Les modèles sont des objets Python, qui définissent la structure des données d'une application, et fournissent des mécanismes de gestion (ajout, modification, suppression) et requêtent les enregistrements d'une base de données.

- **Templates:** Un template est un fichier texte qui définit la structure ou la mise en page d'un fichier (comme une page HTML), avec des balises utilisées pour représenter le contenu. Une *vue* peut créer une page HTML en dynamique en utilisant un template HTML, en la peuplant avec les données d'un *modèle*. Un template peut-être utilisé pour définir la structure de n'importe quel type de fichier; il n'est pas obligatoire que ce dernier soit un HTML !

📌 **Note :** Django mentionne cette organisation sous le nom d'architecture "Modèle Vue Template". Elle a plusieurs similarités avec l'architecture Modèle Vue Contrôleur.

Les sections ci-dessous vous donneront une idée de ce à quoi ressemble ces différentes parties d'une application Django (nous verrons plus de détails plus tard dans le jeu, une fois que nous aurons configuré l'environnement de développement).

Envoyer la requête à la bonne vue (urls.py)

Le mapper URL est généralement stocké dans un fichier nommé **urls.py**. Dans l'exemple ci-dessous, le mapper (`urlpatterns`) définit une liste de mappings entre des *routes* (des *patterns* d'URL spécifiques*) et leur fonction de vue correspondante. Si une requête HTTP est reçue dont l'URL correspond à un pattern spécifié, la fonction vue associée sera alors appelée et passée dans la requête.

PYTHON

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('book/<int:id>/', views.book-detail, name='book-detail'),
    path('catalog/', include('catalog.urls')),
    re_path(r'^([0-9]+)/$', views.best),
]
```

L'objet `urlpatterns` est une liste de fonctions `path()` et/ou `re_path()` (les listes en Python sont définies en utilisant des crochets), où des éléments sont séparés par des virgules et peuvent avoir une [virgule de traîne optionnelle](#). Par exemple: `[item1, item2, item3,]`.

Le premier argument de chaque méthode est une route (pattern) qui sera reconnu. La méthode `path()` utilise des chevrons pour définir les parties de l'URL qui seront capturées et passées dans les fonctions vues comme arguments nommés. La fonction `re_path()` utilise une approche de correspondance de pattern flexible, connue sous le nom d'expression régulière. Nous parlerons de ces dernières dans un prochain article !

Le second argument est une autre fonction qui sera appelée quand le pattern sera reconnu. La notation `views.book-detail` indique que la fonction s'appelle `book-detail()`, et qu'elle se trouve dans un module appelé `views` (i.e. dans un fichier intitulé `views.py`)

Traiter la requête (views.py)

Les vues sont le coeur des applications web. Elles reçoivent des requêtes HTTP de clients web et renvoient des réponses HTTP. Entretemps, elles mobilisent les autres ressources du framework pour accéder aux bases de données, préparer le rendu des templates, etc.

L'exemple ci-dessous montre une fonction vue minimale `index()`, qui pourrait être appelée par notre mapper URL de la section précédente. Comme toutes les fonctions vues, elle reçoit un objet `HttpRequest` comme paramètre (`request`) et renvoie un objet `HttpResponse`. Dans notre cas on ne fait rien de spécial avec la requête; et notre réponse ne renvoie qu'une chaîne de caractères brute. Nous vous montrerons une requête plus intéressante dans une autre section.

PYTHON

```
## nom du fichier : view.py (fonction vue Django)

from django.http import HttpResponse

def index(request):
    # Reçoit une HttpRequest - le paramètre request
    # réalise des opérations en utilisant les informations de la requête
    # Renvoie HttpResponse
    return HttpResponse('Hello from Django!')
```

Note : Un peu de Python :

- Les [modules Python](#) sont des bibliothèques de fonctions, stockés dans des fichiers séparés que l'on peut vouloir utiliser dans notre code. Ici, nous importons l'objet `HttpResponse` du module `django.http` pour qu'on puisse l'utiliser dans notre vue : `from django.http import HttpResponse`. Il y a d'autres façons d'importer quelques objets (ou tous les objets) d'un module.
- Les fonctions sont déclarées en utilisant le mot-clé `def` comme indiqué ci-dessus, avec des paramètres nommés listés entre parenthèses après le nom de la fonction; la ligne se termine ensuite par deux points. Notez que les lignes suivantes sont **indentées**. L'indentation est importante, car elle spécifie que les lignes de code sont contenues dans un bloc particulier (l'indentation obligatoire est un élément clé de Python, et une des raisons pour lesquelles le code Python est si simple à lire).

Les vues sont généralement stockées dans un fichier nommé **views.py**.

Définir les modèles de données (models.py)

Les applications web Django gèrent et requêtent les données via des objets Python appelés modèles. Les modèles définissent la structure des données stockées, ce qui inclut le champ *types* ainsi qu'au besoin leur taille maximum, les valeurs par défaut, les options de listes pouvant être sélectionnées, le texte d'aide pour la documentation — vous pouvez choisir ce dont vous avez besoin par rapport aux spécifications de votre projet. Une fois que vous avez choisi la base de données que vous souhaitez utiliser, vous n'avez pas du tout besoin de communiquer avec elle directement — vous n'avez qu'à écrire la structure de votre modèle, Django s'occupe du sale boulot de la communication avec la base de données pour vous.

L'extrait de code ci-dessous montre un modèle Django très simple pour un objet `Team`. La classe `Team` est dérivée de la classe Django `models.Model`. Elle définit le nom et le niveau de l'équipe comme des chaînes de caractères et elle spécifie le nombre maximum de caractères pouvant être stockés pour chaque enregistrement. Le champ `team_level` peut avoir plusieurs valeurs, donc nous le définissons comme une liste de choix, puis on fournit à la classe un mapping entre les choix qui seront affichés et les données stockées, avec une valeur par défaut.

PYTHON


```
# nom du fichier : models.py

from django.db import models

class Team(models.Model):
    team_name = models.CharField(max_length=40)

    TEAM_LEVELS = (
        ('U09', 'Under 09s'),
        ('U10', 'Under 10s'),
        ('U11', 'Under 11s'),
        ... # lister les autres niveaux d'équipes
    )

    team_level = models.CharField(max_length=3, choices=TEAM_LEVELS, default='U11')
```

 Note : Un peu de Python :

- Python supporte la "programmation orientée-objet", un type de programmation où l'on organise notre code en objets, ce qui inclut les données et fonctions liées qui agiront sur les données. Les objets peuvent être hérités/étendus/dérivés d'autres objets, ce qui permet à ces objets de partager un comportement commun. En Python, on utilise le mot-clé `class` pour définir le "squelette" d'un objet. On peut créer plusieurs *instances* spécifiques de ce type d'objet d'après le modèle d'une classe.

Ainsi par exemple, nous avons ici une classe `Team`, dérivée de la classe `Model`. Cela signifie que c'est un modèle, et qu'elle contiendra toutes les méthodes d'un modèle, mais qu'on peut aussi lui donner des caractéristiques spécifiques. Dans notre modèle, nous définissons les champs dont aura besoin notre base de données, en leur donnant des noms spécifiques. Django utilisera ces définitions, ce qui inclut aussi le nom des champs, pour créer la base de données sous-jacente.

Requêter les données (views.py)

Le modèle Django fournit une API de requête simplifiée qui nous permet de faire des recherches dans une base de données. Cette API peut inclure plusieurs champs à la fois en supportant plusieurs critères (e.g. exactement, insensible à la casse, supérieur à, etc.), et peut supporter des déclarations complexes (par exemple, vous pouvez spécifier une recherche sur les équipes U11 ayant un nom d'équipe commençant par "Fr" ou se terminant par "al").

L'extrait de code ci-dessous montre une fonction vue (gestionnaire de ressources) affichant toutes nos équipes U09. La ligne en gras montre comment on peut utiliser l'API de requête pour filtrer tous les enregistrements où le champ `team_level` comprend strictement le texte 'U09' (notez comment ce critère est passé dans la fonction `filter()` comme argument, où le nom du champ et le type de correspondance sont séparés par un double underscore : `team_level__exact`).

PYTHON

```
## nom du fichier : views.py

from django.shortcuts import render
```

```
from .models import Team

def index(request):
    list_teams = Team.objects.filter(team_level__exact="U09")
    context = {'youngest_teams': list_teams}
    return render(request, '/best/index.html', context)
```

Cette fonction utilise la fonction `render()` pour créer la `HttpResponse` qui est renvoyée au navigateur. Cette fonction est un *raccourci*; elle crée un fichier HTML en combinant un template HTML spécifique et des données à insérer dans le template (fournies dans la variable appelée " `context` "). Dans la prochaine section, nous vous montrons comment des données sont insérées dans le template pour générer le HTML.

Renvoyer les données (templates HTML)

Les systèmes template vous permettent de spécifier la structure d'un document en output, en utilisant des paramètres fictifs qui seront substitués par les données lorsque la page est générée. Les templates sont souvent utilisées pour créer du HTML, mais ils peuvent aussi être utilisées pour créer d'autres types de documents. Django supporte à la fois son système natif de template ainsi qu'une autre librairie Python populaire prête à l'emploi appelée Jinja2 (il peut aussi supporter d'autres systèmes au besoin).

L'extrait de code ci-dessous montre à quoi pourrait ressembler le template HTML de la section précédente une fois appelé par la fonction `render()`. Ce template a été écrit avec l'hypothèse qu'il aurait accès à une liste de variables appelées `youngest_teams` lorsqu'il est généré (contenu dans la variable `context` dans la fonction `render()` ci-dessus). Dans le squelette HTML nous avons une expression qui vérifie tout d'abord que la variable `youngest_teams` existe, puis itère dessus dans une boucle `for`. À chaque itération, le template affiche la valeur du `team_name` de chaque équipe dans un élément `<li>`.

PYTHON

```
## nom du fichier : best/templates/best/index.html

<!DOCTYPE html>
<html lang="en">
<body>

{% if youngest_teams %}
    <ul>
        {% for team in youngest_teams %}
            <li>{{ team.team_name }}</li>
        {% endfor %}
    </ul>
{% else %}
    <p>No teams are available.</p>
{% endif %}

</body>
</html>
```


Que pouvez-vous faire d'autre ?

Les sections précédentes présentent les caractéristiques principales que vous utiliserez dans presque toutes vos applications web : mapping URL, vues, modèles et templates. Parmi les autres caractéristiques offertes par Django, on peut aussi trouver :

- **Formulaires** : Les formulaires HTML sont utilisés pour collecter des données utilisateurs qui seront traitées sur le serveur. Django simplifie la création, la validation et le traitement des formulaires.
- **Authentification et permissions des utilisateurs**: Django inclut un système d'authentification utilisateur et de gestion des permissions robuste créé avec la sécurité comme priorité lors de sa conception.
- **Cache** : Générer du contenu en dynamique demande bien plus de ressources computationnelles (et est plus lent) que de servir du contenu statique. Django fournit un système de cache flexible qui vous permet de stocker toute ou une partie d'une page afin qu'elle ne soit re-générée que lorsque c'est nécessaire.
- **Administration du site** : L'administration du site avec Django est incluse par défaut lorsque vous créez une application en utilisant le squelette de base. Django permet de créer très simplement une page d'administration où les administrateurs peuvent créer, éditer et voir n'importe quel modèle de données sur votre site.
- **Sérialisation des données** : Django permet de simplifier la sérialisation et de servir vos données en XML ou en JSON. Cela peut être utile si vous créez un service web (un site web dont le seul but est de servir des données qui seront utilisées par d'autres applications ou sites, mais n'affiche rien par lui-même), ou quand vous créez un site web où le code côté client s'occupe d'afficher les données.

Sommaire

Félicitations, vous avez atteint la première étape dans votre voyage avec Django ! Vous devriez désormais comprendre les principaux bénéfices de Django, en savoir un peu plus sur son histoire, et grossièrement à quoi ressemblent chaque partie de votre application Django. Vous devriez aussi avoir appris 2-3 choses à propos du langage de programmation Python, ce qui inclut la syntaxe des listes, fonctions et classes.

Vous avez déjà vu un peu de vrai code Django ci-dessus, mais à la différence du code côté client, vous aurez besoin de configurer un environnement de développement pour l'utiliser. C'est notre prochaine étape.

[Aperçu : Django Web Framework \(Python\)](#)

[Suivant →](#)



Votre modèle pour un internet meilleur.